

EXPERIMENT -10

PROGRAM ON TRAFFIC LIGHTS CONTROLLER MECHANISM

AIM: To write an Assembly Language Program to interface a traffic light controller mechanism

TOOLS REQUIRED: PC , Keil μ Vision software

THEORY:

The Traffic Light Controller developed using assembly language is a microcontroller-based system designed to automatically control traffic signals at an intersection. It operates by switching between red, yellow, and green lights in a specific time sequence to regulate vehicle movement safely and efficiently. In this program, an 8051 microcontroller is used, and the control logic is written in assembly language to directly manage the output ports. Different hexadecimal values are sent to the output address (0FFC0H) using the MOVX instruction to represent various light combinations for two directions (e.g., North–South and East–West).

The system uses Timer 0 of the 8051, configured in Mode 1 (16-bit), to generate time delays between each signal change. The delay subroutine loads the timer registers (TH0 and TL0) with initial values, starts the timer, waits until the overflow flag is set, and then repeats the process to produce a consistent delay for each traffic phase. This ensures that every signal (red, yellow, green) remains ON for a specific duration before the next transition occurs.

Overall, this project demonstrates the fundamental principles of embedded control, timing operations, and sequential logic using assembly language. It provides a practical example of how microcontrollers can be programmed to perform real-time automation tasks, forming the foundation for more advanced smart traffic management systems.

TMOD REGISTER:

(MSB)				(LSB)			
GATE	C/T	M1	M0	GATE	C/T	M1	M0
Timer1				Timer0			

MODES OF TIMER:

M0	M1	Mode	Timer Pulses
0	0	M0	13-bit- 2^{13} -8192
0	1	M1	16-bit- 2^{16} -65535 pulses
1	0	M2	8-bit-autoreload mode- 2^8 = 256 pulses
1	1	M3	Split mode(load the values in T0 automatically start the T1)

TRAFFIC CONTROLLER TABLE:

Stage	R1	G1	R2	G2	R3	G3	R4	G4
6A	0	1	1	0	1	0	1	0
9A	1	0	0	1	1	0	1	0
A6	1	0	1	0	0	1	1	0
A9	1	0	1	0	1	0	0	1

THEORETICAL CALCULATIONS:

FOR 1 SEC TIME DELAY

- Since XTAL = 11.0592 MHz
- 12 Machine Cycles = 1 Clock Cycles
- Single clock period = $11.0592\text{MHz}/12 = 921.6\text{KHz}$
 $= 1/921.6 = 1.085\mu\text{s}$
- Required delay = number of clocks X single clock period
- 1sec = number of clocks X $1.085\mu\text{s}$
- Number of clocks = 9,21,65
- Mode 1 is 16 bit timer, Maximum clocks 65536
- Maximum time delay from 0000H state to FFFFH state is= $65536 \times 1.085\mu\text{s} = 71.1\text{ms}$
- 1 sec = loop count X 71.1ms
- Loop count = $14.06 = 0\text{EH}$
- Therefore, we have initial values in TH = #00H and TL = #00H, TMOD = #01H

PROCEDURE:

1. Turn on the computer, create a folder on D drive saved with Register Number.
2. Open Keil μ Vision5 in desktop or windows, start menu > all programs > open Keil uVision5.

Creating Project:

3. Go to project > click on new μ Vision project > create a new folder saved with experiment number within the already existed register number folder in D drive mentioned in step 1. Then enter the project name > click on save.
4. A Dialogue Box will appears with name, Select the device for target. In devices, Enter P89C51RD2XX and click on Ok. Then select **No** for dialog box message “Copy STARTUP.A51 to project folder and add files to project”. (or) Choose NXP > to select the device P89C51RD2XX > click on ok > select **No** for Copy STARTUP.A51 to project folder.

Creating Coding File:

5. Go to file > new > save (choose the path to save the file, It is saved within the name of experiment number folder mentioned in step 3) > enter a file name with extension **.asm** > save the file.

Linking the Coding File to Project :

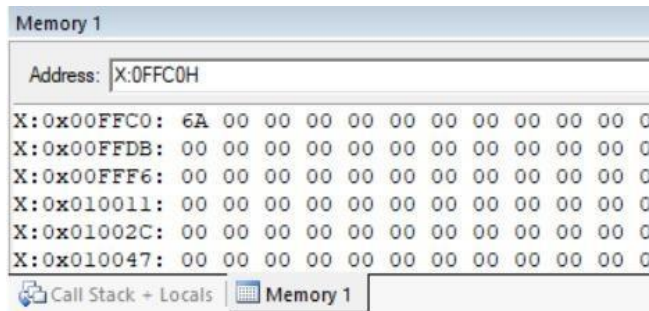
6. Right-click on Source group1 in project bar > Add existing files to source group1 > choose the experiment number folder path and select all files in the folder > select the required .asm code file > add > close.
7. Write the assembly language program in .asm code file and save it.

Executing the Code File:

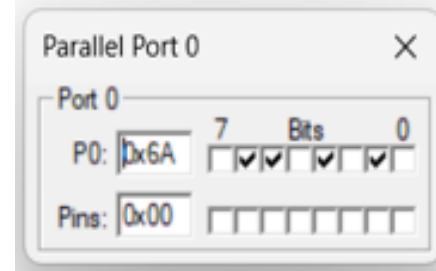
8. Right-click on .asm code file > Build target (to check the errors i.e 0-Errors,0-Warning).
9. Go to debug > Click on Start/Stop Debug Session > click on ok for dialog box message “running code size limit 2K” > and Click on RUN in debug label
10. Observe the output in Register windows, Memory windows, Serial window.

OUTPUT:

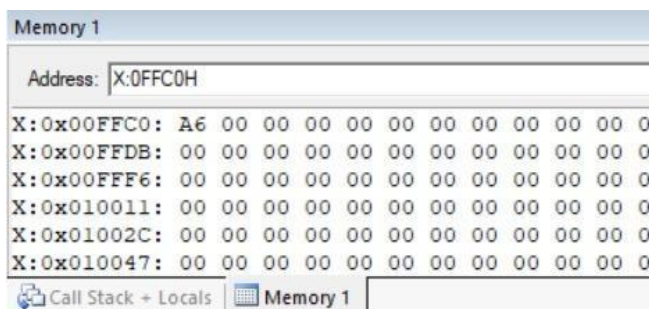
View→memory window→memory1



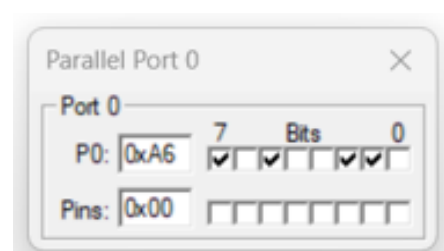
peripherals→I/O-ports→port0



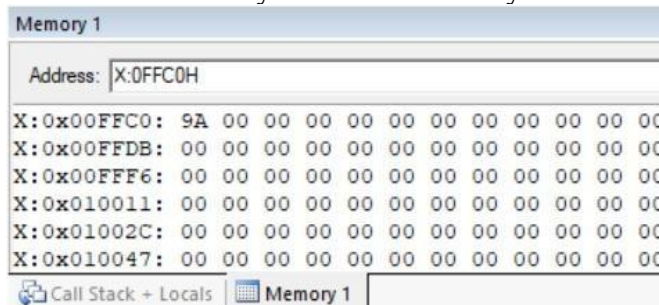
View→memory window→memory1



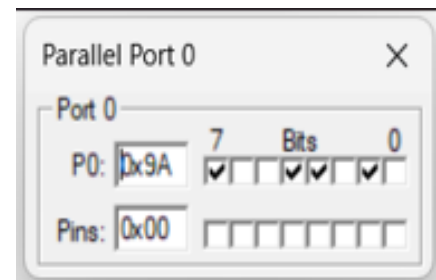
peripherals→I/O-ports→port0



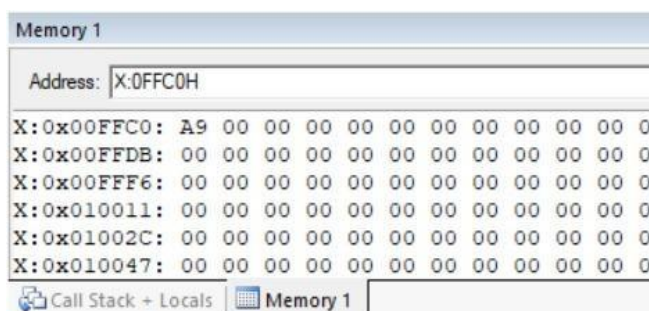
View→memory window→memory1



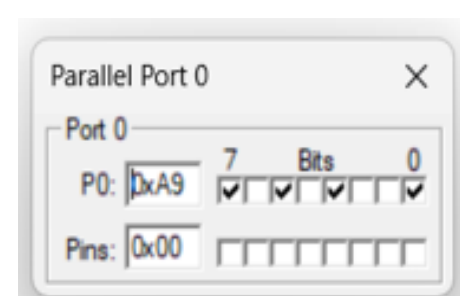
peripherals→I/O-ports→port0



View→memory window→memory1



peripherals→I/O-ports→port0



PROGRAMS:

ADDRESS	OPCODE	LABEL	MNEMONIC	OPERANDS
0000			ORG	0000H
0000	758901		MOV	TMOD,#01H
0003	90FFC0		MOV	DPTR,#0FFC0H
0006	746A	L1:	MOV	A,#6AH
0008	F0		MOVB	@DPTR,A
0009	F580		MOV	P0,A
000B	1124		ACALL	DELAY
000D	749A		MOV	A,#9AH
000F	F0		MOVB	@DPTR,A
0010	F580		MOV	P0,A
0012	1124		ACALL	DELAY
0014	74A6		MOV	A,#0A6H
0016	F0		MOVB	@DPTR,A
0017	F580		MOV	P0,A
0019	1124		ACALL	DELAY
001B	74A9		MOV	A,#0A9H
001D	F0		MOVB	@DPTR,A
001E	F580		MOV	P0,A
0020	1124		ACALL	DELAY
0022	80E2		SJMP	L1
0024	780A	DELAY	MOV	R0,#0AH
0026	758A00	L2	MOV	TL0,#00H
0029	758C00		MOV	TH0,#00H
002C	D28C		SETB	TR0
002E	308DFD	HERE	JNB	TF0,HERE
0031	C28C		CLR	TR0
0033	C28D		CLR	TF0
0035	D8EF		DJNZ	R0,L2
0037	22		RET	
			END	

RESULT: The Assembly Language Program for Traffic light Controller is executed successfully.